

Exercise 1: (6 pts)

A "counter" has a name (3addad for example) and it counts from 1 to n (any positive integer number). It marks a random pause after each number except the last one.

A counter displays each number (3addadwill display for example, "3addad: 7") and it displays a message of type "*** 3addad has finished counting till 10" when it has finished.

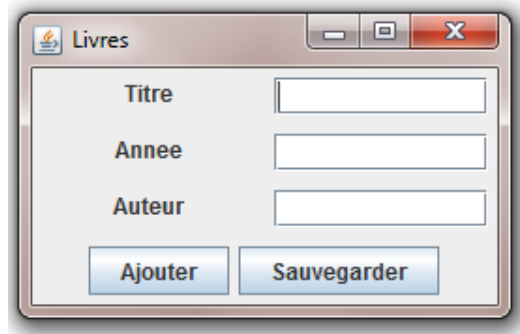
Write the class «Counter» and test it by running many counters that count till 10. Show the counter that has finished first.

Exercise 2: (14 pts)

Consider the following class named *Livre*:

```
public class Livre {  
    private String titre;  
    private int annee;  
    private String auteur;  
  
    public Livre(String titre, int annee, String auteur) {  
        this.titre = titre;  
        this.annee = annee;  
        this.auteur = auteur;  
    }  
    public int getAnnee() {  
        return annee;  
    }  
    public String getAuteur() {  
        return auteur;  
    }  
    public String getTitre() {  
        return titre;  
    }  
}
```

Consider the following graphical interface that allows the data input of a *Livre* using text fields. The button *Ajouter* allows the creation of a new *Livre* object and adds it to a *Vector*. The button *Sauvegarder* allows the saving of the objects *Livre* of the *Vector* to a file. The file's data are loaded at program start-up using the method *charger* below.

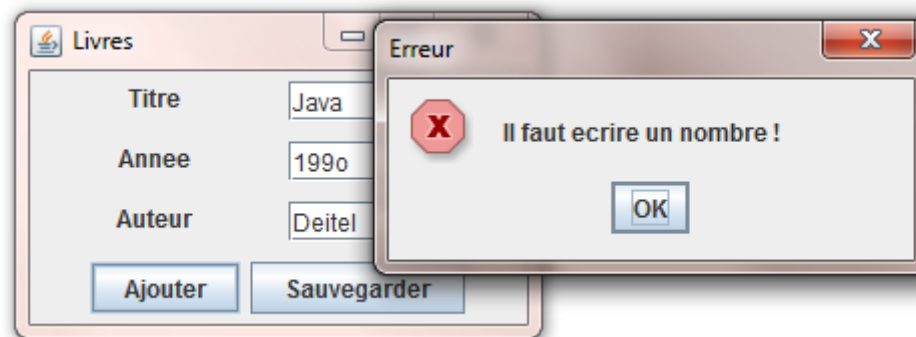


The corresponding code of this graphical interface is as follows:

```
public class MainFrame extends Frame {  
    private TextField titre = new TextField(10);  
    private TextField annee = new TextField(10);  
    private TextField auteur = new TextField(10);  
    private Label lTitre = new Label("Titre");  
    private Label lAnnee = new Label("Annee");  
    private Label lAuteur = new Label("Auteur");  
    private Button ajouter = new Button("Ajouter");  
    private Button sauvegarder = new Button("Sauvegarder");  
    private Vector liste = new Vector();  
  
    public MainFrame() {  
        // to complete ... (1)  
        this.pack();  
        charger();  
        // to complete ... (2)  
    }  
    private void ajouterPerformed() {  
        // to complete ... (3)  
    }  
    private void sauvgarderPerformed() {  
        // to complete ... (4)  
    }  
}
```

```
public void sauvegarder2Performed() {  
    // to complete ... (5)  
}  
private void charger() {  
    // to complete ... (6)  
}  
}
```

- a) Give in (1) the necessary code in order to obtain the above graphical interface. (You may use the layout managers *BorderLayout*, *FlowLayout*, and *GridLayout*).
- b) Give in (2) the code that adds an *ActionListener* to the button *Ajouter* and executes the method *ajouterPerformed*.
- c) Give in (3) the code that allows the data input in the text fields, to create an object *Livre*, and to add it to the object *liste*. The text fields should be empty after the addition.
- d) Give in (4) the code that opens the file « livre.txt » and to write the content of the *liste* to it. Each line must contain the data of one *Livre* object as follows:
Java,2000,Deitel
- e) Give in (5) the code that serializes (persists) the *Livre* objects of the *liste* to a file « data.bin ». Which change should you do to the class *Livre* regarding the persistence?
- f) Give in (6) the code that loads the content of the file « livre.txt », creates the *Livre* objects (one by line), and adds them to the *liste*.
- g) The method *parseInt* raises an exception of type *NumberFormatException* if the String passed as parameter cannot be converted to an integer number. Change the code of the method *ajouterPerformed()* in order to support this exception. The raise of the exception will display a *JOptionPane* as follows:



Exercice 1: (6 pts)

Un "compteur" a un nom (3addad par exemple) et il compte de 1 à n (nombre entier positif quelconque). Il marque une pause aléatoire après chaque nombre sauf le dernier.

Un compteur affiche chaque nombre (3addad affichera par exemple, "3addad: 7") et il affiche un message du type "*** 3addad a fini de compter jusqu'à 10" quand il a fini.

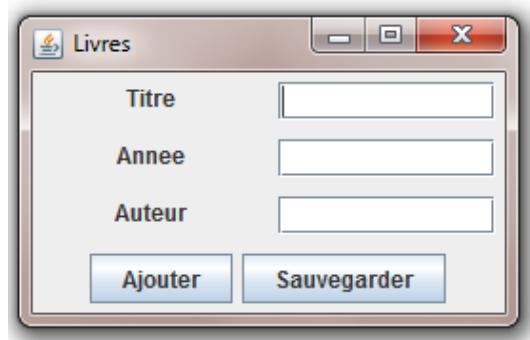
Écrire la classe « Compteur » et tester-la en lançant plusieurs compteurs qui comptent jusqu'à 10. Montrer celui qui a fini le premier.

Exercice 2: (14 pts)

On considère la classe *Livre* suivante:

```
public class Livre {  
    private String titre;  
    private int annee;  
    private String auteur;  
  
    public Livre(String titre, int annee, String auteur) {  
        this.titre = titre;  
        this.annee = annee;  
        this.auteur = auteur;  
    }  
    public int getAnnee() {  
        return annee;  
    }  
    public String getAuteur() {  
        return auteur;  
    }  
    public String getTitre() {  
        return titre;  
    }  
}
```

On considère l'interface graphique suivante permettant de saisir les données d'un livre dans des champs de texte. Le bouton *Ajouter* permet de créer un nouvel objet *Livre* et de l'ajouter à un *Vector*. Le bouton *Sauvegarder* permet de sauvegarder les livres du *Vector* dans un fichier. Les données du fichier sont chargées au démarrage du programme à l'aide de la méthode *charger* ci-dessous.



Le code correspondant à cette interface graphique est comme suit:

```
public class MainFrame extends Frame {  
    private TextField titre = new TextField(10);  
    private TextField annee = new TextField(10);  
    private TextField auteur = new TextField(10);  
    private Label lTitre = new Label("Titre");  
    private Label lAnnee = new Label("Annee");  
    private Label lAuteur = new Label("Auteur");  
    private Button ajouter = new Button("Ajouter");  
    private Button sauvegarder = new Button("Sauvegarder");  
    private Vector liste = new Vector();  
  
    public MainFrame() {  
        // à compléter ... (1)  
        this.pack();  
        charger();  
        // à compléter ... (2)  
    }  
    private void ajouterPerformed() {  
        // à compléter ... (3)  
    }  
    private void sauvegarderPerformed() {  
        // à compléter ... (4)  
    }  
}
```

```
public void sauvegarder2Performed() {  
    // à compléter ... (5)  
}  
private void charger() {  
    // à compléter ... (6)  
}  
}
```

- a) Donner en (1) le code nécessaire afin d'obtenir l'interface graphique ci-dessus. (Vous pouvez utiliser les gestionnaires *BorderLayout*, *FlowLayout*, et *GridLayout*).
- b) Donner en (2) le code qui permet d'ajouter un écouteur d'action (*ActionListener*) au bouton *Ajouter* et qui exécute la méthode *ajouterPerformed*.
- c) Donner en (3) le code permettant de saisir les champs de texte, de créer un objet *Livre*, et de l'ajouter à l'objet *liste*. Les champs doivent être vides après l'ajout.
- d) Donner en (4) le code permettant d'ouvrir le fichier « livre.txt » et d'y écrire les informations des livres de la *liste*. Chaque ligne doit contenir les informations d'un seul livre comme suit :
Java,2000,Deitel
- e) Donner en (5) le code permettant de sérialiser (persister) les objets *Livre* de la *liste* dans un fichier « data.bin ». Quel changement doit-on effectuer à la classe *Livre* pour la persistance?
- f) Donner en (6) le code permettant de charger les informations contenues dans le fichier « livre.txt », de créer des objets *Livre* (un par ligne) et de les ajouter à la *liste*.
- g) La méthode *parseInt* soulève une exception du type *NumberFormatException* si la String passée en paramètre ne peut pas être convertie en un nombre entier. Changer le code de la méthode *ajouterPerformed()* afin de supporter cette exception. L'appel de cette exception affichera un *JOptionPane* comme suit:

